

MSML 605

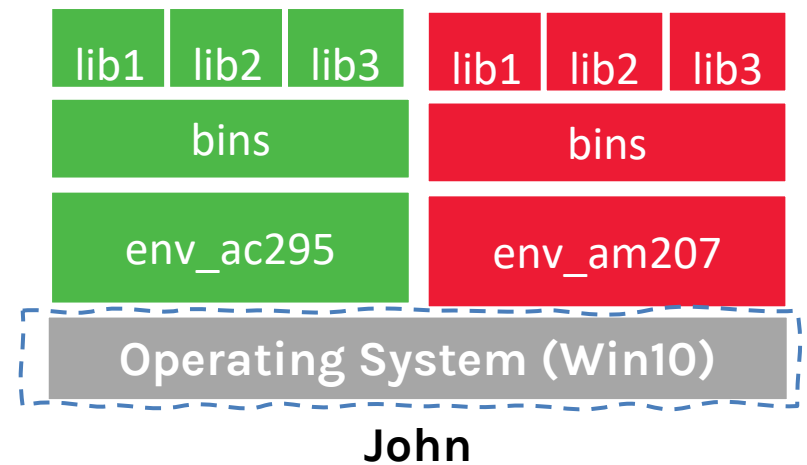
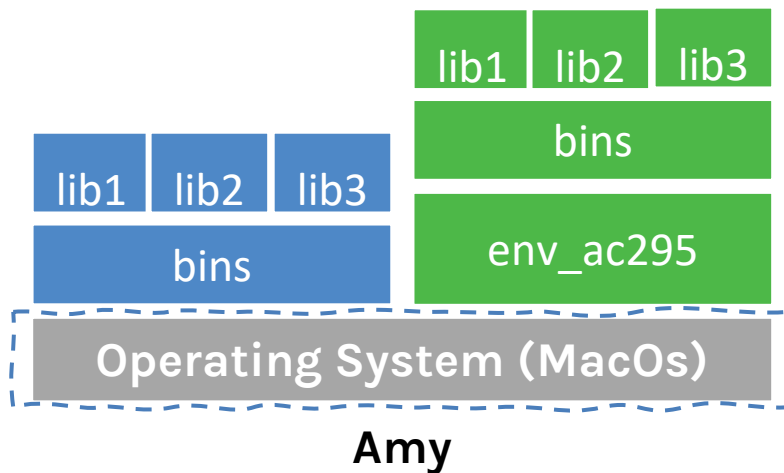
Containers



docker

Why should we use virtual environment?

- What could go wrong? Unfortunately, Amy and John reproduce different results, and they think the issue relates to their operating systems. Indeed, while Amy has a MacOS, John uses a Win10.



Virtual environments

Pros

- Reproducible research
- Explicit dependencies
- Improved engineering collaboration
- Broader skill set

Cons

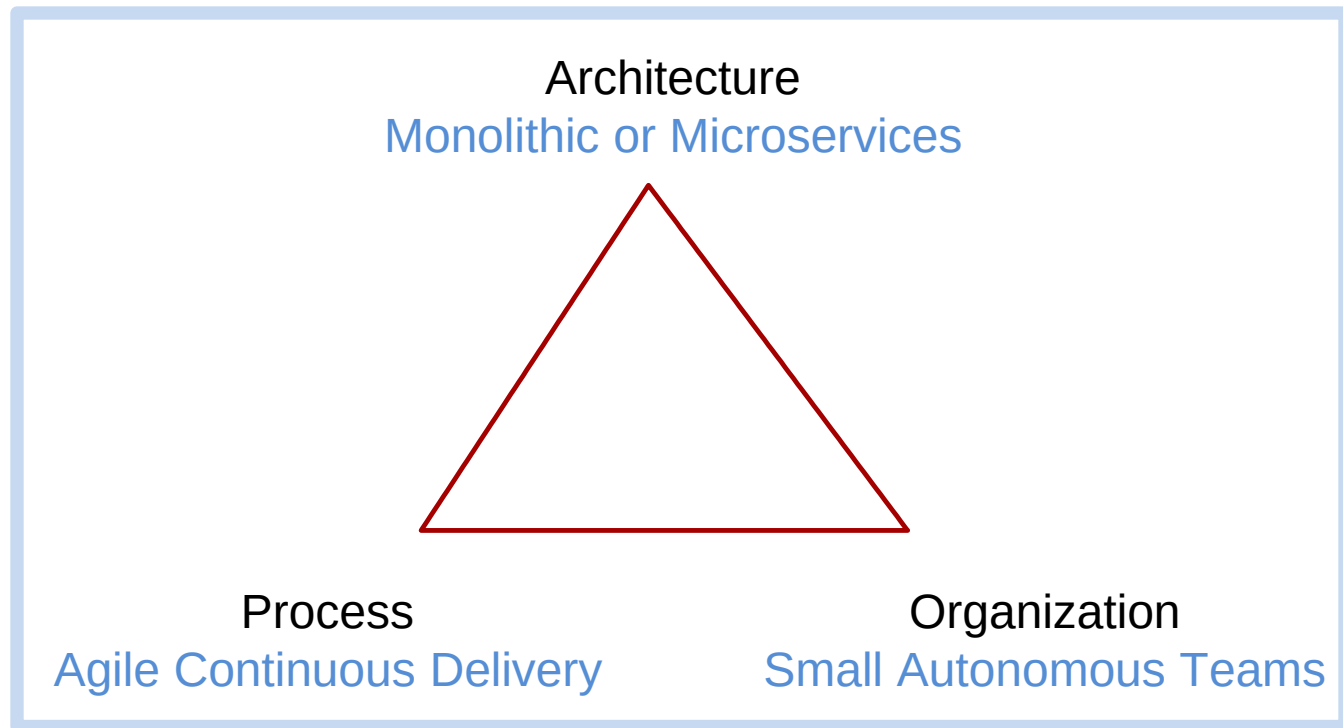
- Difficulty setting up your environment
 - Not isolation
 - Does not work across different OS
-

Limitations

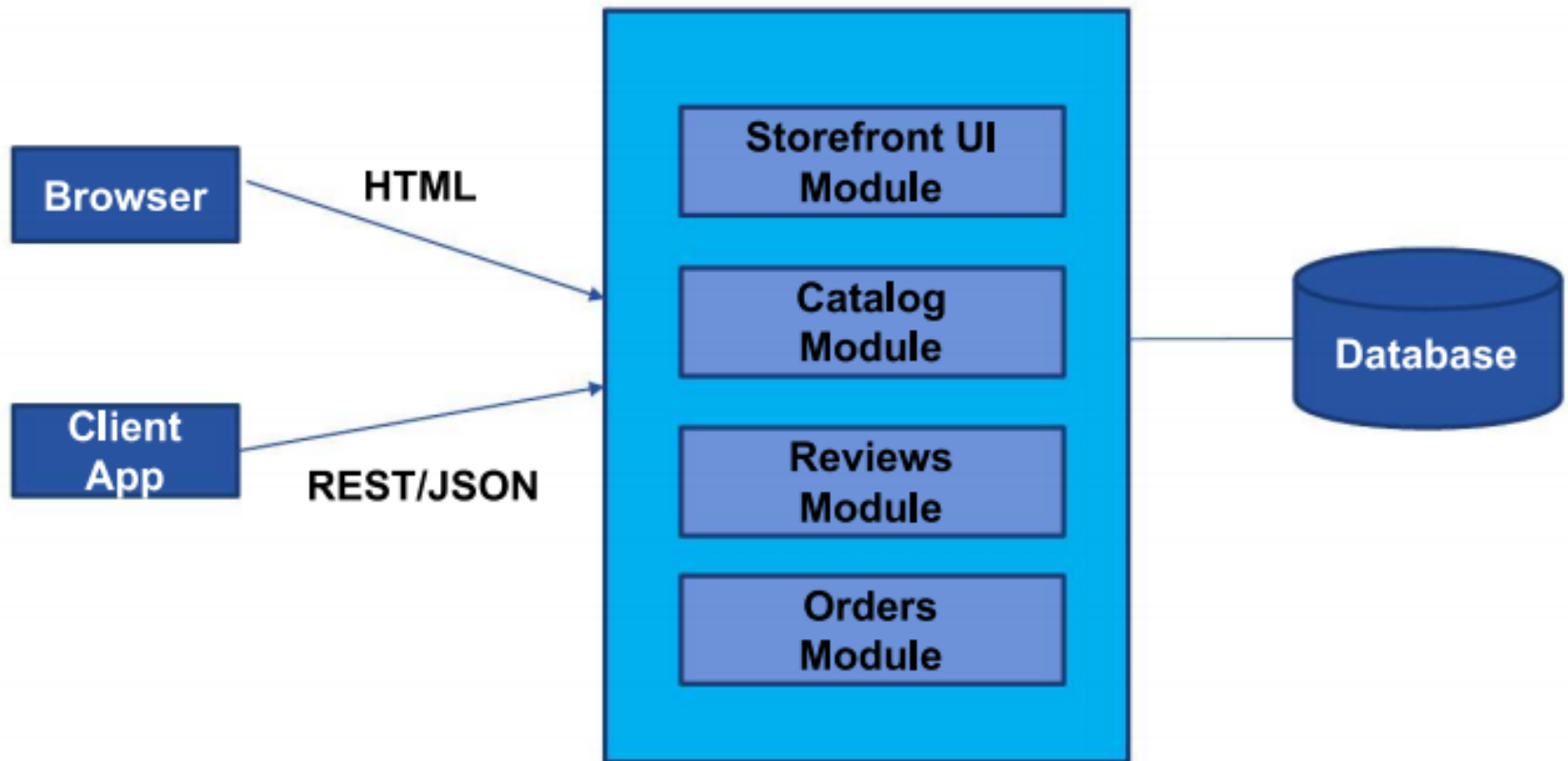
- Uses hardware in your local machine (cannot run more than two on an average laptop)
 - There is overhead associated with virtual machines
 1. guest is not as fast as the host system
 2. Takes long time to start up
 3. may not have the same graphics capabilities
-

Successful Software Application

- Imagine you are building a large complex application (e.g. Online Store)



Monolithic Architecture



Advantages of Monolith

Simple to **Develop**, **Test**, **Deploy** and **Scale**:

- Simple to develop because all the tools and IDEs support the applications by default
 - Easy to deploy because all components are packed into one bundle
 - Easy to scale the whole application
-

Disadvantages of Monolith

- Very difficult to maintain
 - One component failure will cause the whole system to fail
 - Very difficult to create the patches for monolithic architecture
 - Adapting to new technologies is challenging
 - Takes a long time to startup because all the components needs to get started
-

Evolution of Applications

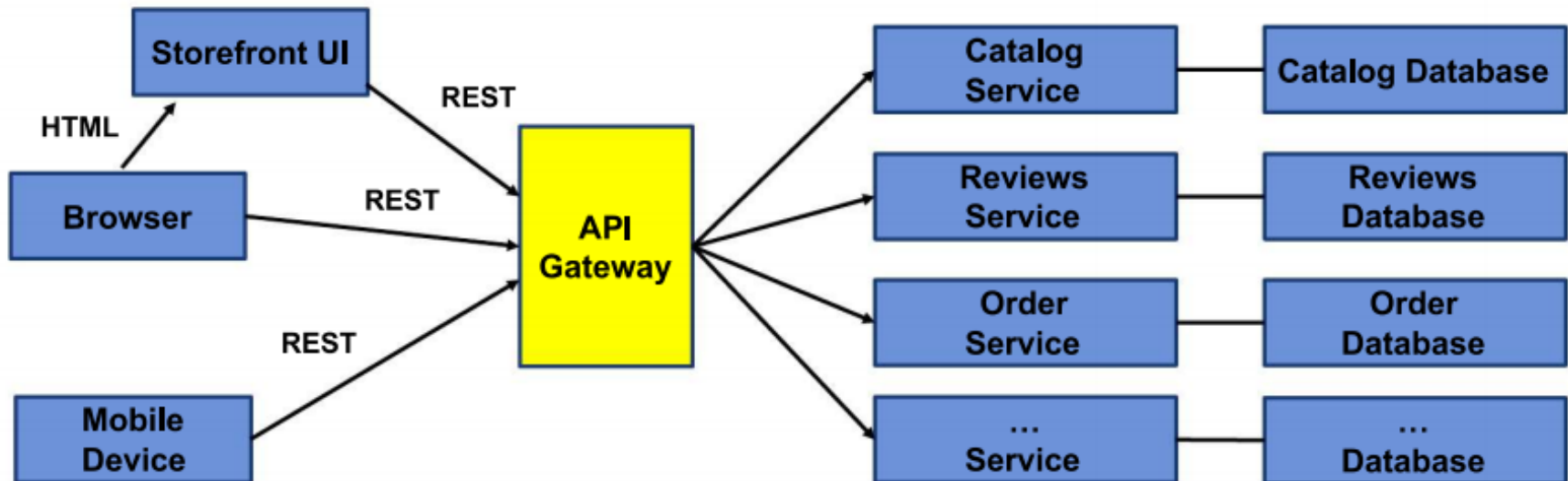
A decade ago

- Apps were monolithic
- Built on a single stack (.NET or Java)
- Long lived
- Deployed to a single server

Today

- Apps are constantly being developed
- Build from loosely coupled components
- Newer version are deployed often
- Deployed to a multitude of servers

Microservices Architecture



What is a Container?



- Standardized packaging for software dependencies
- Isolates apps from one another
- Works for all major Linux distributions, MacOS, Windows

Docker

- A container encapsulates software and its dependencies.
 - Docker is an open-source platform for developing, shipping and running applications as containers.
 - It is based on Linux containers and written in Go programming language.
 - Uses a namespace for an isolated workspace called a container
-

Docker

- Build an app once and run anywhere
 - Containers take fewer resources
 - Results in stable application deployment
 - Deployments happen in seconds
-

Docker Properties

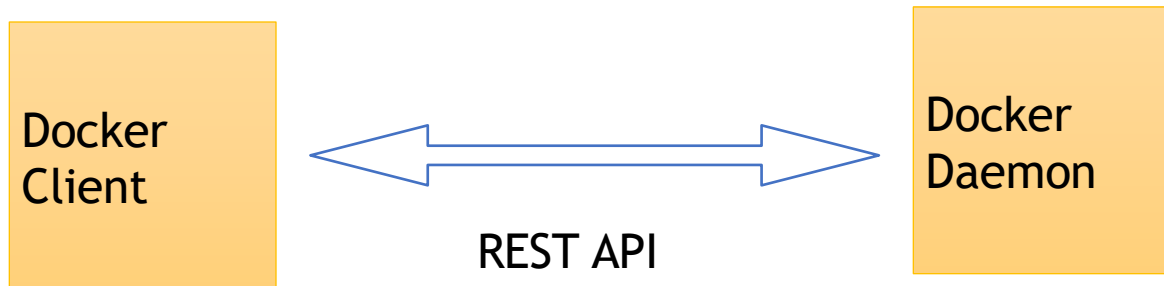
- Containerization: Enables creation and management of lightweight, portable containers that package applications and their dependencies.
 - Image-based. Docker images are read-only templates containing instructions for creating a container.
 - Portability: Can run consistently across platforms, including local development environments, including local development machines, data centers and cloud platforms.
 - Efficient: Shares host computers kernel and resources.
-

Docker Properties

- Versioning & layering: Enables developers to track changes, rollback, and optimize image reuse.
 - Orchestration: Docker provides tools for container management for multi-container management. Kubernetes has become standard for managing containers
 - Security: Provides built-in security for protection of containerized applications, including, namespace isolation
 - Dockerhub: Active community, hosts pre-built docker images
-

Docker Engine

- To do anything with docker you need to install docker engine.
- Get an OS specific image from <https://docs.docker.com/get-docker/>



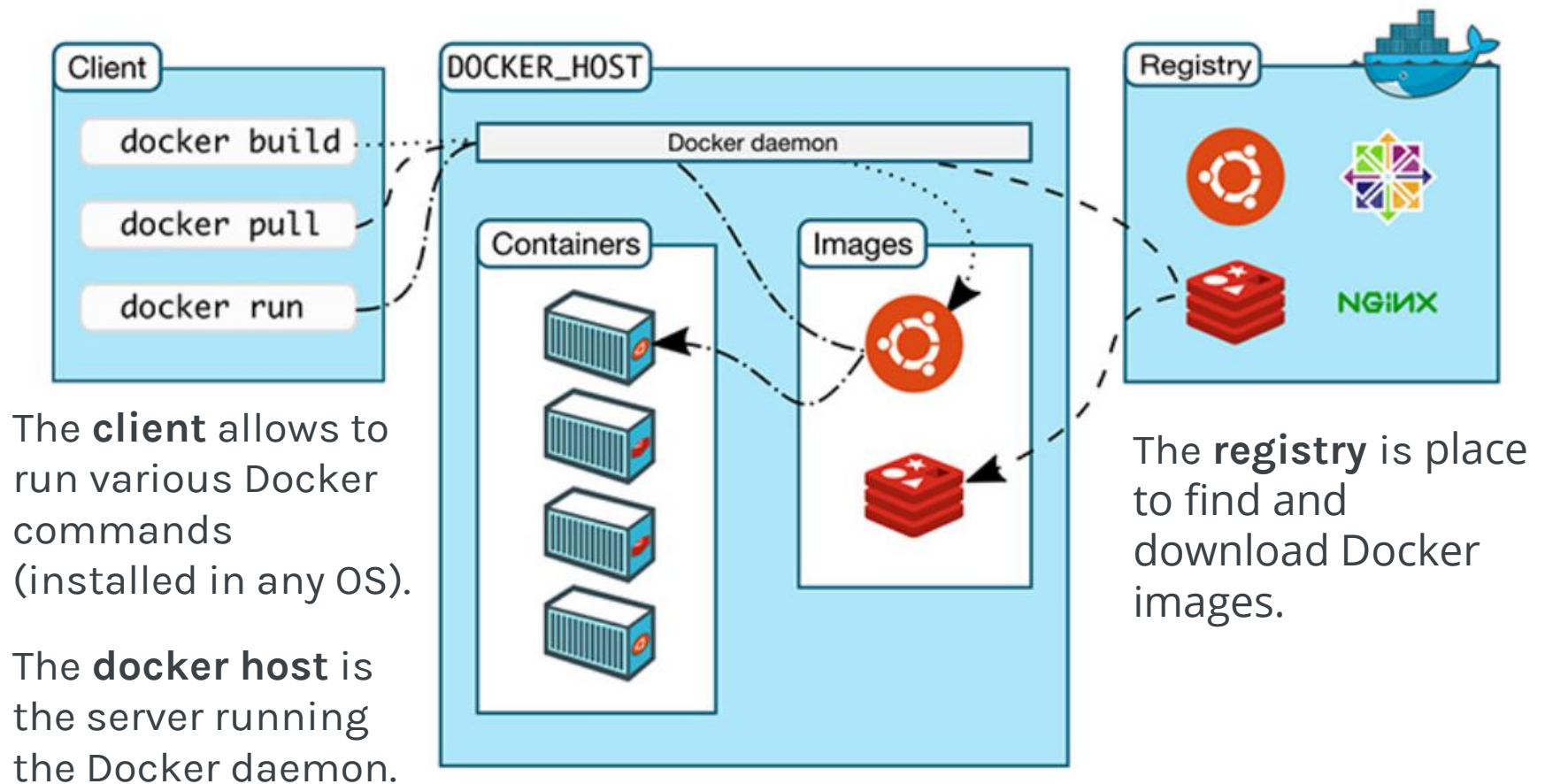
- To check that it is installed: `docker —version`

Docker Engine

- To check that it is installed: *docker —version*

```
Docker version 20.10.11, build dea9396
```

Docker Engine Architecture



<https://nickjanetakis.com/blog/understanding-how-the-docker-daemon-and-docker-cli-work-together>

Docker Client

- It is the user interface
 - You communicate with it.
 - Docker client can connect to the server on the local host as well as a remote host
 - It communicates with the Docker Daemon
-

Docker Daemon

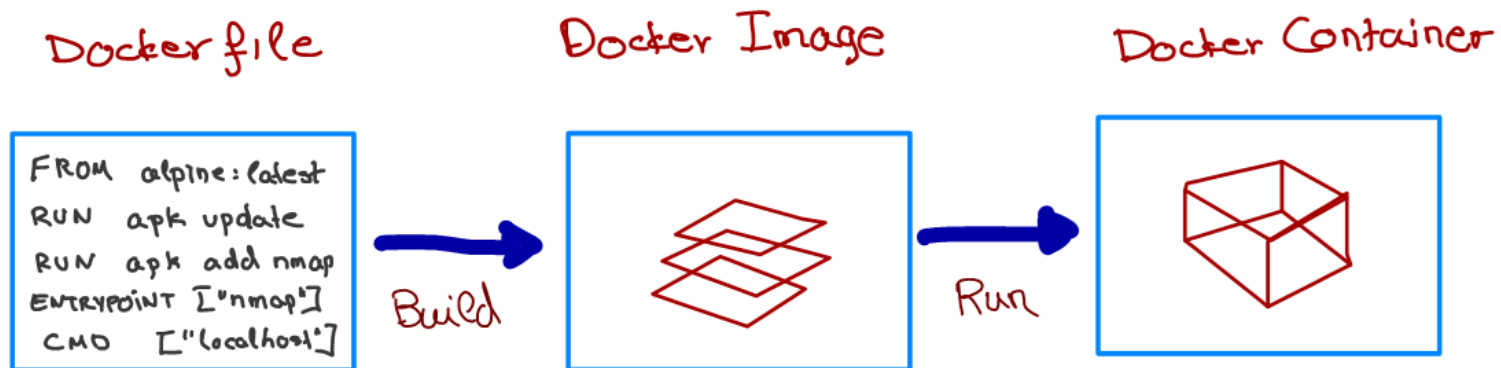
- It executes the commands you send to the client.
- For example, building, running, and distributing your containers.

Image vs Container

- Docker Image is a template aka blueprint to create a running Docker container. Docker uses the information available in the Image to create (run) a container.
 - Image is like a recipe, container is like a dish
 - You can think of an image as a class and a container is an instance of that class.
-

Steps to create and run containers

- Create a Docker file
- Use that Docker file to create a container image.
- Use the container image to create a running image.



Dockerfile

- It contains the instructions to build a Docker image.
 - For example, to install a software, set environmental variables etc.
 - Use *docker build* command to build an *image* from it.
 - Docker uses a Union file system.
 - Once you run the image it is a *container*
-

Inside the Dockerfile

FROM: tells the daemon, which base image to use while creating our new Docker image. In the example, we are using a very minimal OS image called alpine.

RUN: This command instructs the Docker daemon to run the given commands as it is while creating the image. A Dockerfile can have multiple RUN commands, each of these RUN commands create a new **layer** in the image.

ENTRYPOINT: The ENTRYPOINT instruction is used when you would like your container to run the same executable every time. Usually, ENTRYPOINT is used to specify the binary and CMD to provide parameters.

CMD: The CMD sets default command and/or parameters when a docker container runs. **CMD can be overwritten** from the command line via the docker run command.

Dockerfile

```
FROM alpine:latest
RUN apk update
RUN apk add nmap
ENTRYPOINT ["nmap"]
CMD ["localhost"]
```


Multiple containers from same image

- How can you run multiple containers from the same image? Wouldn't they all be identical?
- Think of an image as instantiating a class. You could instantiate it with different parameters using the CMD and therefore different containers will be different.

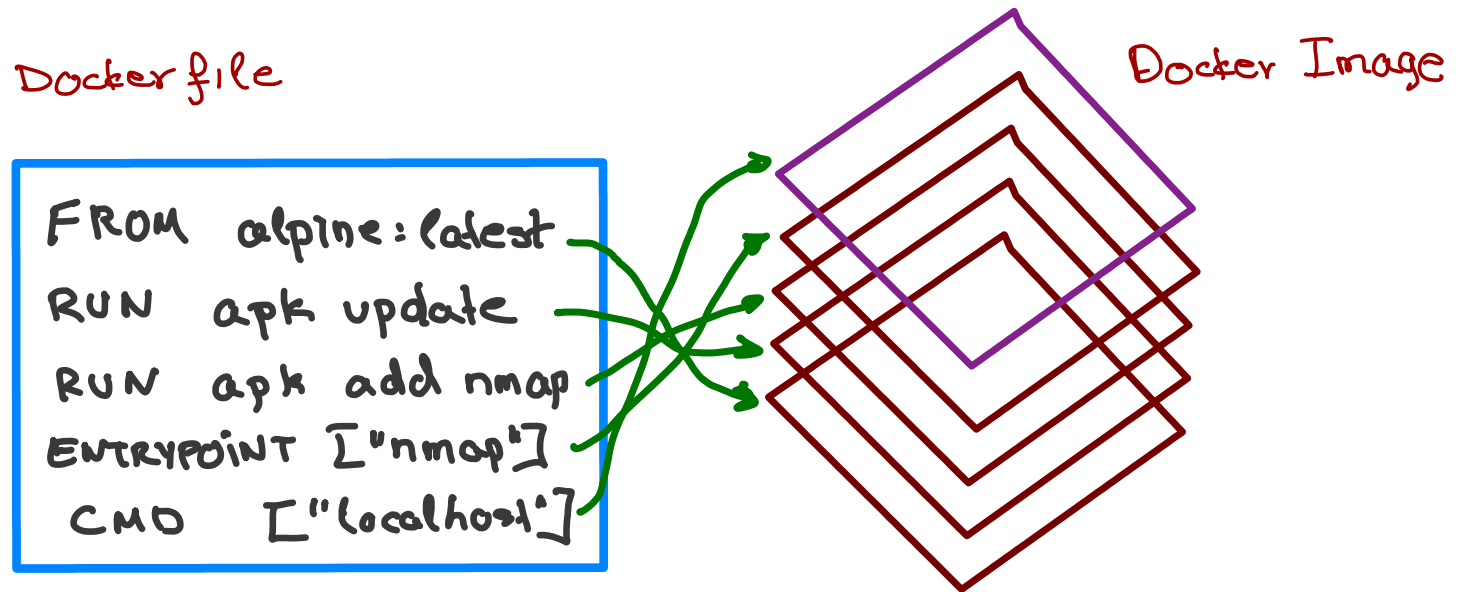
Dockerfile

```
FROM ubuntu:latest  
RUN apt-get update  
ENTRYPOINT ["/bin/echo", "Hello"]  
CMD ["world"]
```

```
> docker build -t hello_world_cmd:first -f  
Dockerfile_cmd .  
  
> docker run -it hello_world_cmd:first  
> Hello world  
> docker run -it hello_world_cmd:first  
MSML605  
> Hello MSML605
```

Docker Image as Layers

- When we execute the build command, the daemon reads the **Dockerfile** and creates a **layer** for each command.



Docker Project - First dockerApp

- *docker images - list the images (docker image is read-only image containing instructions for creating docker container)*

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my0app	v1	fd003a252205	3 weeks ago	7.73MB
my0app	v2	fd003a252205	3 weeks ago	7.73MB
<none>	<none>	6b08d185715d	3 weeks ago	7.73MB
<none>	<none>	1e9921e6d5a4	3 weeks ago	7.73MB
<none>	<none>	ae0ad7e18d82	3 weeks ago	7.73MB
<none>	<none>	44b10aba2867	3 weeks ago	7.73MB
firstapp	v1	9c5a4f8198d8	3 weeks ago	7.73MB
firstapp	v0	587befc65d68	3 weeks ago	7.73MB
my0app	v3	587befc65d68	3 weeks ago	7.73MB

- *docker run - create container from the image* (runnable instance of an image)
docker run firstapp:v1

```
Hello MSML 605!
```

Docker Project - dockerApp

- Python file: *code.py*

```
import numpy as np
```

```
print('Numpy version:', p.__version__)
```

```
arr = np.random.rand(10) # Random array of size 10
```

```
print("Generated Array:",arr)
```

```
# Calculate and print the mean of the array
```

```
print("Mean of the array: ",np.mean(arr))
```

- *Requirements.txt* pipreqs

```
numpy==1.24.0
```

Dockerfile

```
# Use the official Python image as a base image
FROM python:3.9
```

```
# Set the working directory within the container
WORKDIR /app
```

```
# Copy the requirements file into the container at /app
COPY requirements.txt /app
```

```
# Install the required Python packages
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy the current directory contents into the container at /app
COPY . /app
```

```
# Run the Python script when the container launches
CMD ["python", "code.py"]
```

Dockerfile

```
# Use the official Python image as a base image
FROM python:3.9

# Set the working directory within the container
WORKDIR /app

# Copy the requirements file into the container at /app
COPY requirements.txt /app

# Install the required Python packages
RUN pip install --no-cache-dir -r requirements.txt

# Copy the current directory contents into the container at /app
COPY . /app

# Run the Python script when the container launches
CMD ["python", "code.py"]
```

- FROM: get a parent image from the docker hub
Each command in the docker file uses this parent image.
- Copy requirements file into the app folder

Dockerfile

```
# Use the official Python image as a base image
FROM python:3.9

# Set the working directory within the container
WORKDIR /app

# Copy the requirements file into the container at /app
COPY requirements.txt /app

# Install the required Python packages
RUN pip install --no-cache-dir -r requirements.txt

# Copy the current directory contents into the container at /app
COPY . /app

# Run the Python script when the container launches
CMD ["python", "code.py"]
```

- Set app as the working directory
- Run pip commands :
 - install all the dependencies in *requirements.txt* in the parent image.

Dockerfile

```
# Use the official Python image as a base image  
FROM python:3.9
```

```
# Set the working directory within the container  
WORKDIR /app
```

```
# Copy the requirements file into the container at /app  
COPY requirements.txt /app
```

```
# Install the required Python packages  
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy the current directory contents into the container at /app  
COPY . /app
```

```
# Run the Python script when the container launches  
CMD ["python", "code.py"]
```

- Copy the app, *code.py* to the app
- Define what command gets executed.

Docker Image

```
docker build -t pythoncodeapp .
```

```
[+] Building 0.2s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from dockerfile                0.0s
=> => transferring dockerfile: 499B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9      0.2s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.9@sha256:383d072c4b840507f25453c710969aa1e1d13e47731f294a8a8890e53f834bdf 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 94B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                        0.0s
=> CACHED [3/5] COPY requirements.txt /app                          0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . /app                                         0.0s
=> exporting to image                                              0.0s
=> => exporting layers                                              0.0s
=> => writing image sha256:28622b5f192564c7ae7ccda653bd91db08cd1ae1de017f32bdf6ecd3769fe34e 0.0s
=> => naming to docker.io/library/pythoncodeapp                    0.0s
```

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/nmqgg50ctkj7khnfnai2mdana

What's Next?

View a summary of image vulnerabilities and recommendations → [docker scout quickview](#)

List all local images

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
pythoncodeapp	latest	28622b5f1925	8 minutes ago	1.06GB
<none>	<none>	cbb82e2fc4fc	8 minutes ago	1.06GB
<none>	<none>	fadc3fe10ec5	11 minutes ago	1GB
runapp	v0	48e4a0ff1550	3 hours ago	7.73MB
<none>	<none>	9f45a2327676	3 hours ago	7.73MB
nginx	latest	760b7cbba31e	5 days ago	192MB
<none>	<none>	eb51df36367a	2 weeks ago	1GB
firstapp	v1	9c5a4f8198d8	3 weeks ago	7.73MB
<none>	<none>	69aec704f483	3 weeks ago	7.73MB
my0app	v1	fd003a252205	3 weeks ago	7.73MB
my0app	v2	fd003a252205	3 weeks ago	7.73MB
<none>	<none>	ae0ad7e18d82	3 weeks ago	7.73MB
<none>	<none>	44b10aba2867	3 weeks ago	7.73MB
<none>	<none>	1e9921e6d5a4	3 weeks ago	7.73MB
firstapp	v0	587befc65d68	3 weeks ago	7.73MB
my0app	v3	587befc65d68	3 weeks ago	7.73MB
<none>	<none>	6b08d185715d	3 weeks ago	7.73MB

Remove containers and images

- Remove stopped containers

```
docker container prune
```

- Remove all dangling images

```
docker image prune
```

- Remove all images without at least one container associated to them

```
docker image prune -a
```

Docker hub

- There is a container registry at Docker hub
 - 1 free private repository
 - Go to <https://hub.docker.com/>
 - Create a username
-

Build a docker Image locally

```
docker build -t pythoncodeapp:latest .
```

```
[+] Building 0.4s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from dockerfile                0.0s
=> => transferring dockerfile: 499B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9      0.4s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                      0.0s
=> [1/5] FROM docker.io/library/python:3.9@sha256:383d072c4b840507f25453c710969 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 827B                                    0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt /app                         0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . /app                                        0.0s
=> exporting to image                                              0.0s
=> => exporting layers                                             0.0s
=> => writing image sha256:28622b5f192564c7ae7ccda653bd91db08cd1aelde017f32bdf6 0.0s
=> => naming to docker.io/library/pythoncodeapp:latest           0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/z88dvq
veu6sgz2trh82m4arj0
```

Tag and push to docker hub

- Tag the Docker image

```
docker tag pythoncodeapp:latest nayeemz/pythoncodeapp:latest
```

- Push the Docker Image to Docker Hub

```
docker push nayeemz/pythoncodeapp:latest
```

```
[The push refers to repository [docker.io/nayeemz/pythoncodeapp]  
bc5105565cf7: Pushed  
ab1b06748a92: Pushed  
5732849bb45d: Pushed  
8e5e0ec6f417: Pushed  
3b48824bd4fd: Mounted from library/python  
5589e8997c0c: Mounted from library/python  
5f895c7ab7df: Mounted from library/python  
9ce63ba53cb8: Mounted from library/python  
d9c6bbb693ea: Mounted from library/python  
a974964b27e5: Mounted from library/python  
973599cf2dad: Mounted from library/python  
b10a49b17ae6: Mounted from library/python  
latest: digest: sha256:d40894339c766c0e31f322b529d104520d02b4df81b43b572d68720f575959ae size: 2839
```

Pull

- Pull the Docker image

```
docker pull nayeemz/pythoncodeapp:latest
```

- Run the Docker Image

```
docker run nayeemz/pythoncodeapp
```

Requirements file

- Dependency manager — *pipenv*
- Uses *pip* and *virtualenv*
- *For user installation*

```
pip install --upgrade setuptools wheel
```

```
pip install --user pipenv
```

Set up the PATH

- vi ~/.bash_profile

```
export PATH="/Users/nayeem/.local/bin:$PATH"
```

- *In the terminal*

```
source ~/.bash_profile
```

```
echo $PATH | tr ':' '\n'
```

pipenv

pipenv

```
[(base) Mohammads-MBP:~ nayeem$ pipenv  
Usage: pipenv [OPTIONS] COMMAND [ARGS]...
```

Options:

<code>--where</code>	Output project home information.
<code>--venv</code>	Output virtualenv information.
<code>--py</code>	Output Python interpreter information.
<code>--envs</code>	Output Environment Variable options.
<code>--rm</code>	Remove the virtualenv.
<code>--bare</code>	Minimal output.
<code>--completion</code>	Output completion (to be eval'd).
<code>--man</code>	Display manpage.
<code>--support</code>	Output diagnostic information for use in GitHub issues.
<code>--site-packages</code>	Enable site-packages for the virtualenv. [env var: PIPENV_SITE_PACKAGES]
<code>--python TEXT</code>	Specify which version of Python virtualenv should use.
<code>--three / --two</code>	Use Python 3/2 when creating virtualenv.
<code>--clear</code>	Clears caches (pipenv, pip, and pip-tools). [env var: PIPENV_CLEAR]
<code>-v, --verbose</code>	Verbose mode.
<code>--pypi-mirror TEXT</code>	Specify a PyPI mirror.
<code>--version</code>	Show the version and exit.
<code>-h, --help</code>	Show this message and exit.

pipenv - shell to install packages

```
[(base) Mohammads-MBP:dockerApp nayeem$ pipenv shell
Creating a virtualenv for this project...
Pipfile: /Users/nayeem/Documents/UMD/MSML605/Code/dockerApp/Pipfile
Using /anaconda3/bin/python (3.7.3) to create virtualenv...
:: Creating virtual environment...created virtual environment CPython3.7.3.final.
0-64 in 686ms
  creator CPython3Posix(dest=/Users/nayeem/.local/share/virtualenvs/dockerApp-tl
Fthxfm, clear=False, global=False)
  seeder FromAppData(download=False, pip=latest, setuptools=latest, wheel=latest
, via=copy, app_data_dir=/Users/nayeem/Library/Application Support/virtualenv/se
ed-app-data/v1.0.1)
  activators BashActivator,CShellActivator,FishActivator,PowerShellActivator,Pyt
honActivator,XonshActivator

✓ Successfully created virtual environment!
Virtualenv location: /Users/nayeem/.local/share/virtualenvs/dockerApp-tlFthxfm
requirements.txt found, instead of Pipfile! Converting...
✓ Success!
```

pipenv - to install packages

```
(dockerApp) bash-3.2$ pipenv install pandas  
Installing pandas...  
Adding pandas to Pipfile's [packages]...  
✓ Installation Succeeded  
Pipfile.lock not found, creating...  
Locking [dev-packages] dependencies...  
Locking [packages] dependencies...  
✓ Success!  
Updated Pipfile.lock (0b6fa6)!  
Installing dependencies from Pipfile.lock (0b6fa6)...  
 9/9 - 00:00:09  
(dockerApp) bash-3.2$ pipenv install numpy  
Installing numpy...  
Adding numpy to Pipfile's [packages]...  
✓ Installation Succeeded  
Installing dependencies from Pipfile.lock (0b6fa6)...  
 9/9 - 00:00:06
```

```
pipenv run python code.py
```

Numpy version: 1.17.4

Docker Container vs Virtual Machine

VMs

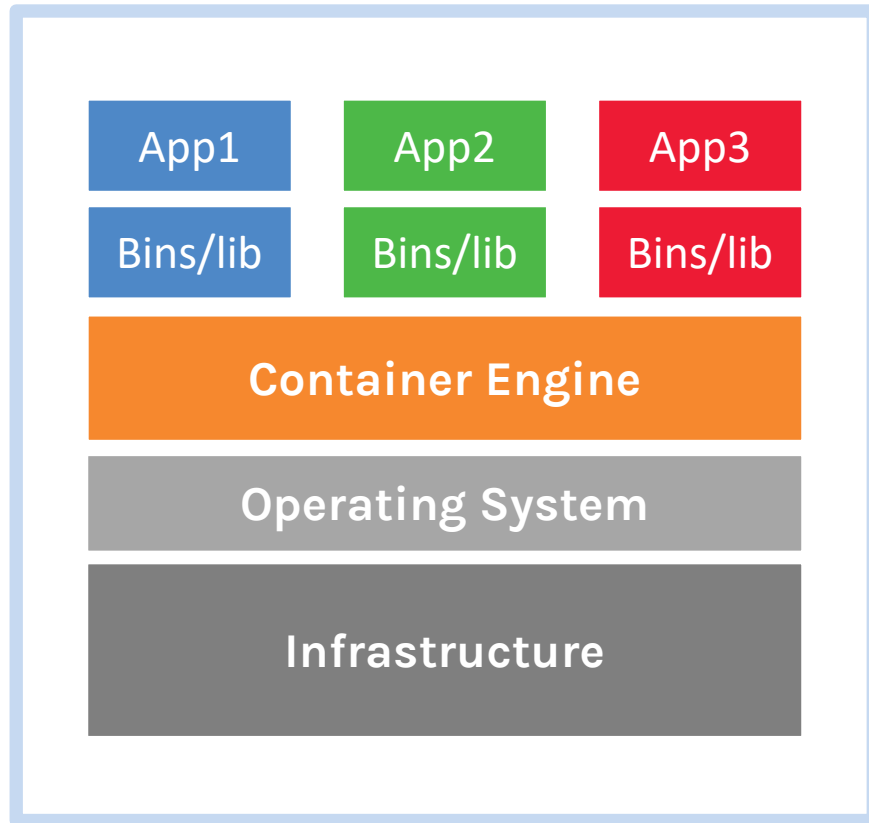
- Each VM runs its own OS
- Boot up time is in minutes
- Not version controlled
- Cannot run more than couple of VMs on an average laptop
- Only one VM can be started from one set of VMX and VMDK files

Docker

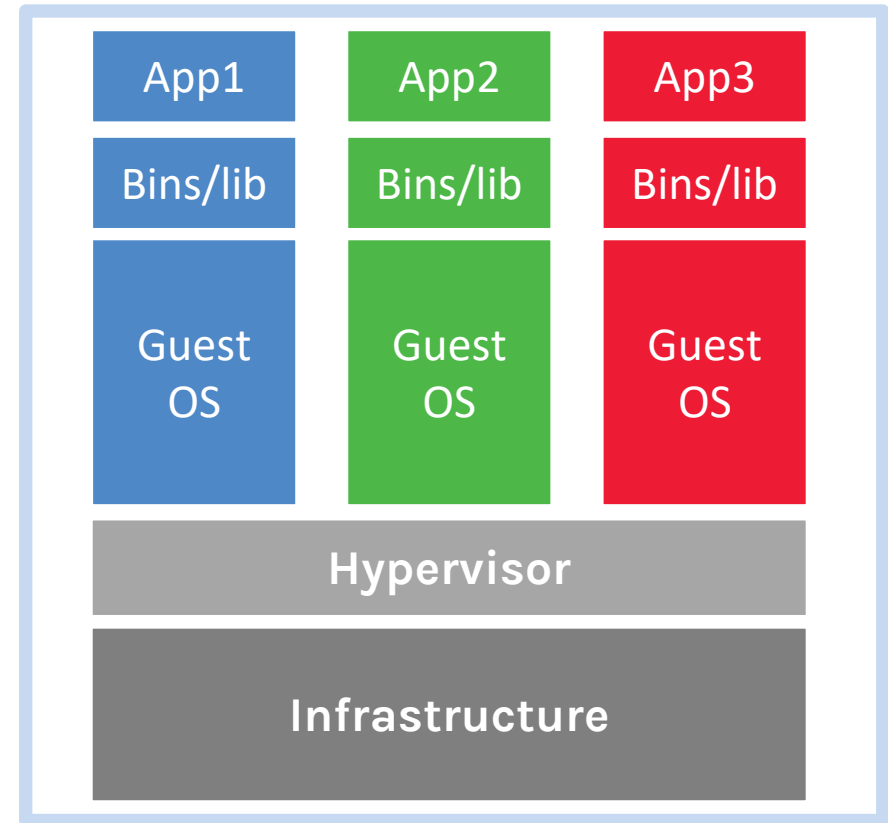
- Container is just a user space of OS
- Containers instantiate in seconds
- Images are built incrementally on top of another like layers. Lots of images/snapshots
- Images can be diffed and can be version controlled. Docker hub is like Github
- Can run many Dockers in a laptop
- Multiple docker containers can be started from one Docker image

Docker Container vs Virtual Machine

Container



Virtual Machine



Why containers?

- Their **startup** time is on the order of seconds (vs. minutes for Virtual Machines).
 - They provide **pseudo-isolation**. This means they're still secure, but not as secure as Virtual Machines.
 - A container is deployed from the container image offering an isolated **executable environment** for the application.
 - Containers can be deployed from a specific image on **many platforms**, such as workstations, Virtual Machines, public cloud, etc.
 - Containers are extremely **popular**, and their popularity is growing.
-

Why containers cont'd?

- One of the first widely used containers was provided by [Docker](#).
 - [Docker](#) containers can be used to run websites and web applications.
 - Multiple containers can be managed by a service called Kubernetes.
-

Hands-on Docker Exercise

- Download the Docker Tutorial from ELMS/Canvas
- Follow the instructions